

Lab Session #9

Generative Models

Dr. Zied M'NASRI

1 Introduction

In this Lab session, we learn how to implement and train some generative models seen in Lecture9, such as the (feedforward) Deep Autoencoder and the Convolutional Autoencoder and how to use them to reconstruct data, e.g. image from original or noisy input.

You will need to use:

1. Python and Colab/Jupyter tutorials (see Lab sessions 1-3 material)
2. Lecture 9 notes
3. Lab 9 code example (*AI_Lab9_Example_Autoencoder.ipynb*) (see the zip file)
4. [Google's Colab](#) or Jupyter Notebook

2 Tutorial - Simple Autoencoder

Upload the notebook *AI_Lab9_Example_Autoencoder.ipynb* to Google Colab and run it, to understand how to design the encoder, the decoder and the full simple/shallow autoencoder.

In this example, add the following commands to view the encoder, decoder and autoencoder models

1. Add the command `model.summary()` to each of the sections `#Encoder model`, `#Decoder model` and `#Full Autoencoder model` and display the architecture of each network, e.g.

```
#Encoder model:
# This model maps an input to its encoded representation
encoder = keras.Model(input_img, encoded)
encoder.summary()
```

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 784)	0
dense_2 (Dense)	(None, 32)	25,120

Total params: 25,120 (98.12 KB)
 Trainable params: 25,120 (98.12 KB)
 Non-trainable params: 0 (0.00 B)

Figure 1. Encoder network

2. After training is finished, visualise the loss curves during training and validation

```
import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label="Training loss")
plt.plot(history.history['val_loss'], label="Validation loss")
plt.title("Loss curves")
plt.legend(loc = "upper left")
plt.show()
```

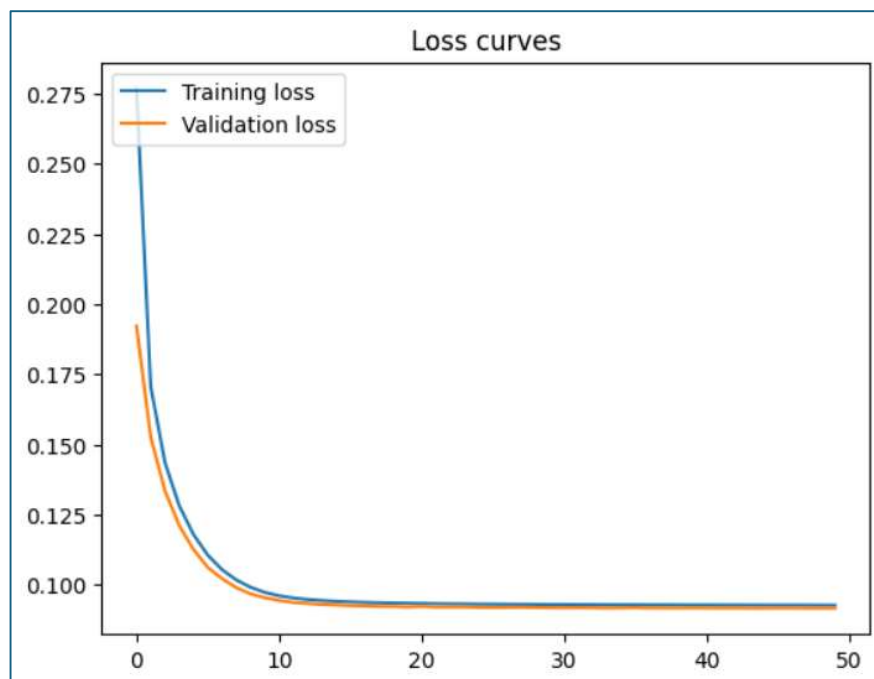


Figure 2. Loss curves on the training and validation sets

3. After the last section, “Reconstruction”, add the following code to visualize some of the original and encoded-decoded images from the test set

```
# Display the original and the encoded-decoded images
import matplotlib.pyplot as plt

n = 10 # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
```

```

ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)

# Display reconstruction
ax = plt.subplot(2, n, i + 1 + n)
plt.imshow(decoded_imgs[i].reshape(28, 28))
plt.gray()
ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)
plt.show()

```

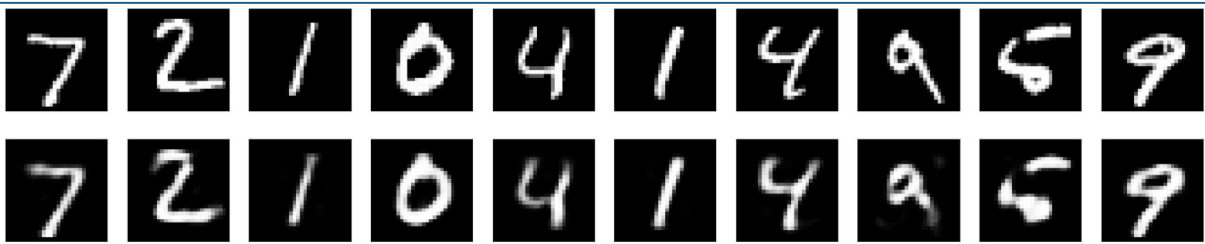


Figure 3. Original and reconstructed (encoded-decoded) images from MNIST dataset

4. Since we only reconstruct the input data (images), the evaluation metrics consists of the reconstruction error, which can be measured by the Root Mean Squared Error (RMSE) and the relative RMSE between the original and the reconstructed images in the test set.

To calculate RMSE and Relative RMSE, add the following code at the end

```

from sklearn.metrics import mean_squared_error
RMSE = np.sqrt(mean_squared_error(x_test, decoded_imgs))
Relative_RMSE = RMSE/np.mean(x_test)
print("Root Mean Square Error: ", RMSE)
print("Relative MSE: ", Relative_RMSE)

```

```

Root Mean Square Error: 0.098067194
Relative MSE: 0.7400478

```

Figure 4. Evaluation metrics

3 Exercises

3.1 Exercise #1 - (Feedforward) Deep Autoencoder

In this exercise we aim at designing and training a deep autoencoder model for MNIST dataset. The main difference with the simple autoencoder (see the Warm-up Example above) is the architecture of the encoder and decoder networks, where several feedforward layers are stacked.

Steps

1. Setup

Import the necessary libraries

```
#Setup
import keras
from keras import layers
from keras import regularizers
from json import decoder
from keras.datasets import mnist
import numpy as np
```

2. Encoder network

Build the Encoder network following these steps

- 2.1. Specify the input layer as `keras.input` with `shape=(784,)`
- 2.2. Build the first encoded image (encoded) where the input is `input_img` and the output is encoded:

```
encoded = layers.Dense(128, activation='relu')(input_img)
```

- 2.3. Add another Dense layer with 64 nodes and 'relu' activation, where the input is encoded and the output is also encoded

- 2.4. In the same way, add a third Dense layer with 32 nodes and 'relu' activation. This will be the output layer of the encoder

Hint! The input is encoded and output is also encoded

- 2.5. Build the encoder network using the method `keras.model()` with the arguments `input_img` and `encoded` as input and output, respectively.

- 2.6. Visualize the encoder network using the method `encoderModelName.summary()`

Model: "functional"		
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
Total params: 110,816 (432.88 KB)		
Trainable params: 110,816 (432.88 KB)		
Non-trainable params: 0 (0.00 B)		

Figure 1.1. Encoder network

3. Decoder network

In the same way, build the decoder network, using the following settings:

- 3.1. Create a variable named `decoded` to receive the output of the decoder. `decoded` will be the output of the first Dense layer with 32 nodes and `'relu'` activation. The input is the variable named `encoded` (i.e. the result of the encoder)
- 3.2. Add a second Dense layer with 64 nodes and `'relu'` activation, where the output and the input are also decoded and encoded, respectively.
- 3.3. Add the third and final Dense layer with 128 nodes and `'relu'` activation (with the same input and output, i.e. encoded and decoded).
- 3.4. Define the decoder network as `decoderModelName` using the method `keras.model()` and `encoded` and `decoded` as arguments.
- 3.5. Display the created decoder network using the method `decoderModelName.summary()`

Model: "functional_12"		
Layer (type)	Output Shape	Param #
keras_tensor_19CLONE (InputLayer)	(None, 32)	0
dense_18 (Dense)	(None, 64)	2,112
dense_19 (Dense)	(None, 128)	8,320
dense_20 (Dense)	(None, 784)	101,136
Total params: 111,568 (435.81 KB)		
Trainable params: 111,568 (435.81 KB)		
Non-trainable params: 0 (0.00 B)		

Figure 1.2. Decoder network

4. Full autoencoder network

4.1. Now, build the full autoencoder network by simply applying the method `keras.model()` and the variables `input_img` and `decoded` as arguments.

4.2. Display the full autoencoder network using the method `autoencoderModelName.summary()`

Model: "functional_14"		
Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 784)	0
dense_15 (Dense)	(None, 128)	100,480
dense_16 (Dense)	(None, 64)	8,256
dense_17 (Dense)	(None, 32)	2,080
dense_21 (Dense)	(None, 64)	2,112
dense_22 (Dense)	(None, 128)	8,320
dense_23 (Dense)	(None, 784)	101,136
Total params: 222,384 (868.69 KB)		
Trainable params: 222,384 (868.69 KB)		
Non-trainable params: 0 (0.00 B)		

Figure 1.3. Full autoencoder network

5. MNIST dataset

5.1. Upload the MNIST dataset using the method `mnist.load_data()`. In the output, mention only the names of the training and test images (labels are not needed)

5.2. Normalize the pixel values between 0 and 1 by dividing on the maximum pixel value, i.e. 255

5.3. Flatten the images into a 1-D vector and reshape `x_train` and `x_test` into (number of images, 28x28) size

Hint! See Warm-up example.

```
Trainig set shape: (60000, 784)
Test set shape: (10000, 784)
```

Figure 1.4. Size of training and test sets after flattening

6. Configuration and training

6.1. Compile the created autoencoder network using the method `autoencoderModelName.compile()` with the following arguments:
`optimizer='adam', loss='binary_crossentropy'`

6.2. Train the autoencoder model using the method `autoencoderModelName.fit()` with the following arguments:

```
Input: x_train
Output: x:train
Epochs= 50
Batch_size = 512
Shuffle = True
Validation_data = (x_test, x_test)
```

```
Epoch 1/10
235/235 ————— 6s 17ms/step - loss: 0.3473 - val_loss: 0.1692
Epoch 2/10
235/235 ————— 6s 22ms/step - loss: 0.1601 - val_loss: 0.1391
Epoch 3/10
235/235 ————— 9s 15ms/step - loss: 0.1372 - val_loss: 0.1271
Epoch 4/10
235/235 ————— 6s 20ms/step - loss: 0.1270 - val_loss: 0.1192
Epoch 5/10
235/235 ————— 4s 15ms/step - loss: 0.1194 - val_loss: 0.1136
```

Figure 1.5. Training of the autoencoder

7. Reconstruction

When training is finished, use the autoencoder model to reconstruct the images of the test set by using one of both methods:

7.1. Apply the encoder model the input images (`x_test`) to obtain the encoded images, then use the decoder model to decode them

7.2. Apply directly the autoencoder model to reconstruct the input images

Hint! Use the method `modelName.predict()` in each case

7.3. Using the same code as in the Example above, visualise some of the original and reconstructed images



Figure 1.6. Original and reconstructed images by the deep autoencoder

3.2 Exercise #2 - Convolutional Autoencoder

Using the same steps of Exercise #1, define the encoder and decoder networks as follows:

1. Encoder network

1.1. Define the input layer with `keras.input()` and input shape (28,28,1)

Hint! Name the output of this layer as `input_img`

1.2. Add the following layers:

- Conv2D with 16 x (3, 3) filters, activation='relu' and padding='same'
- MaxPooling2D with a (2, 2) filter and padding='same'
- Conv2D with 8 x (3, 3) filters, activation='relu' and padding='same'
- MaxPooling2D with a (2, 2) filter and padding='same'
- Conv2D with 8x (3, 3) filter, activation='relu' and padding='same'
- MaxPooling2D with a (2, 2) filter and padding='same'

Hint! The input to the first layer is `input_img` and the output of the layer is named `encoded`

1.3. Build the encoder network using the method `keras.model()` using `input_img` and `encoded` as arguments.

1.4. Display the encoder model network using the method `encoderModelName.summary()`

Model: "functional_1"		
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 8)	1,160
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 8)	0
conv2d_2 (Conv2D)	(None, 7, 7, 8)	584
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 8)	0
Total params: 1,904 (7.44 KB)		
Trainable params: 1,904 (7.44 KB)		
Non-trainable params: 0 (0.00 B)		

Figure 2.1. Convolution encoder network

2. Decoder network

2.1. Now define the decoder network using the following layers:

- **Input layer:** Conv2D with 8 x (3, 3) filters , activation='relu' and padding='same'
- UpSampling2D with a (2, 2) filter (the reverse operation of the Maxpooling layer)!
- Conv2D with 8 x (3, 3) filters , activation='relu' and padding='same'
- UpSampling2D with a (2, 2) filter
- Conv2D with 8 x (3, 3) filters , activation='relu' and padding='same'
- UpSampling2D with a (2, 2) filter
- **Output layer :** Conv2D with 1x (3, 3), activation='sigmoid' and padding='same'

Hint! At the input layer, the input variable is encoded (result of the encoder), and at output layer, the result is a variable named decoded

2.2. Display the decoder network using the method `decoderModelName.summary()`

Model: "functional_2"		
Layer (type)	Output Shape	Param #
keras_tensor_6CLONE (InputLayer)	(None, 4, 4, 8)	0
conv2d_3 (Conv2D)	(None, 4, 4, 8)	584
up_sampling2d (UpSampling2D)	(None, 8, 8, 8)	0
conv2d_4 (Conv2D)	(None, 8, 8, 8)	584
up_sampling2d_1 (UpSampling2D)	(None, 16, 16, 8)	0
conv2d_5 (Conv2D)	(None, 14, 14, 16)	1,168
up_sampling2d_2 (UpSampling2D)	(None, 28, 28, 16)	0
conv2d_6 (Conv2D)	(None, 28, 28, 1)	145
Total params: 2,481 (9.69 KB)		
Trainable params: 2,481 (9.69 KB)		
Non-trainable params: 0 (0.00 B)		

Figure 2.2. Convolutional decoder network

3. Full autoencoder network

3.1. Build the full convolutional autoencoder network (CAE) using the method `keras.model()` and `input_img` and `decoded` as arguments.

3.2. Display the full CAE network using the method `autoencoderModelName.summary()`

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 8)	1,160
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 8)	0
conv2d_2 (Conv2D)	(None, 7, 7, 8)	584
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 8)	0
conv2d_3 (Conv2D)	(None, 4, 4, 8)	584
up_sampling2d (UpSampling2D)	(None, 8, 8, 8)	0
conv2d_4 (Conv2D)	(None, 8, 8, 8)	584
up_sampling2d_1 (UpSampling2D)	(None, 16, 16, 8)	0
conv2d_5 (Conv2D)	(None, 14, 14, 16)	1,168
up_sampling2d_2 (UpSampling2D)	(None, 28, 28, 16)	0
conv2d_6 (Conv2D)	(None, 28, 28, 1)	145

Total params: 4,385 (17.13 KB)
 Trainable params: 4,385 (17.13 KB)
 Non-trainable params: 0 (0.00 B)

Figure 2.3. Full convolutional autoencoder network

4. MNIST data

- 4.1. Import and normalise the MNIST dataset (see Example or Exercise #1)
- 4.2. Reshape the `x_train` and `x_test` subsets into (number of images, 28, 28)

```
x_train size: (60000, 28, 28)
x_test size: (10000, 28, 28)
```

Figure 2.4. Size of training and test sets after reshaping

5. Configuration and training

Perform configuration and training of the full autoencoder network using the same steps as in Exercise #1

6. Test and visualisation

- 6.1. Predict the output of the encoder network using the method `encoderModelName.predict()`, and display some encoded images of the test set. You will see the result of encoding as follows:

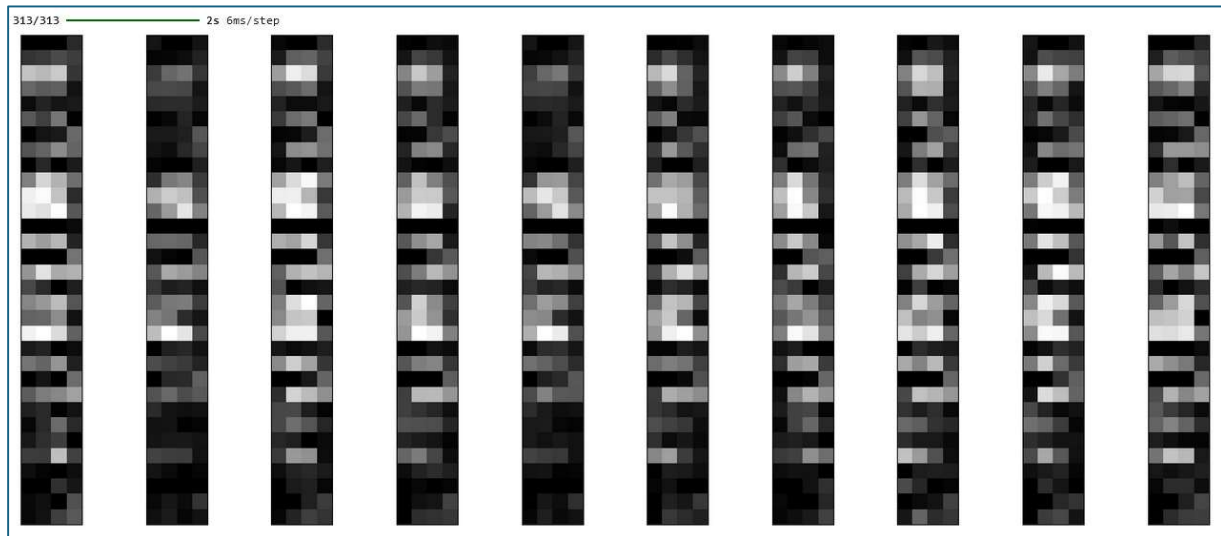


Figure 2.5. Examples of encoded images

6.2. Predict the output of the decoder, using the encoded images as input, and visualise some of their results

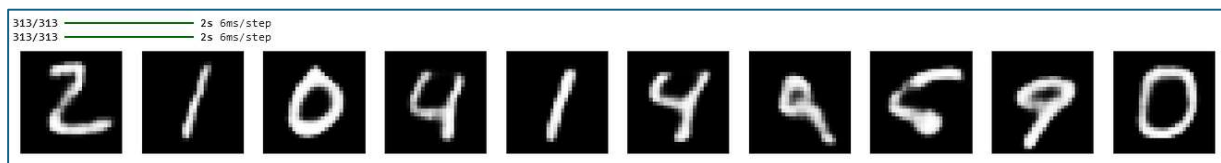


Figure 2.6. Examples of decoded images

3.3 Exercise #3 - Image denoising using Convolutional Autoencoder (CAE)

In this exercise, we aim at using the CAE to enhance the quality of noisy images. To do so, re-take **Exercise #2** and follow these steps:

1. In the data section, after uploading MNIST dataset, normalisation and reshaping `x_train` and `x_test`, add some noise to each subset using:

1.1. Create a random vector `noise_train` (respectively `noise_test`) using the function `np.random.normal()` with the following arguments: `loc=0.0`, `scale=1.0`, `size=x_train.shape` (respectively `x_test.shape` for `noise_test`)

1.2. Multiply the created noise by a factor `noise_factor = 0.5`

1.3. Add the created noise `noise_train` to `x_train` (respectively add `noise_test` to `x_test`)

1.4. To stay in the range of normalised value, i.e. `[0,1]`, clip the result (`x_train_noisy`, respectively `x_test_noisy`) using the method `np.clip()` using `x_train_noisy`, `minValue=0`, `maxValue=1` as arguments.

1.5. Visualise the results of some noisy images from the training set



Figure 3.1. Examples of MNIST images after adding a random normal noise

2. Train the convolutional autoencoder using `x_train_noisy` as input and the clean images `x_train` as output (target)

3. After training, apply the CAE model on `x_test_noisy`, and visualise some examples of noisy images and their output

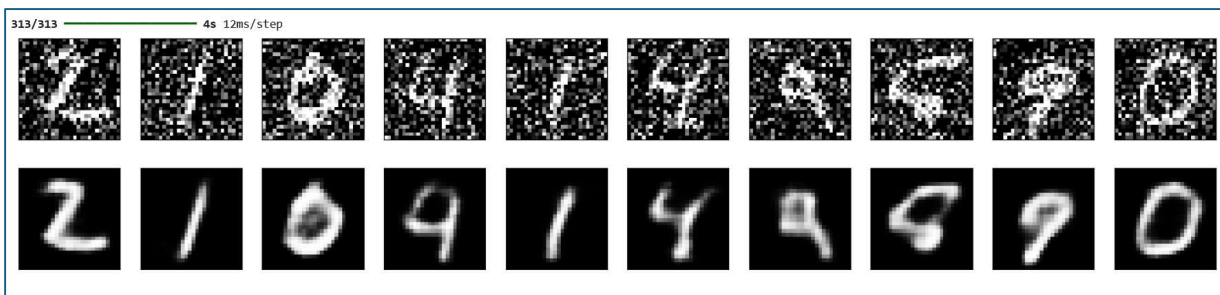


Figure 3.2. Examples of noisy images from the test and their output using the convolutional autoencoder (CAE)